

Gestão de configuração vs. DEVOPS

Você vai compreender o conceito de DevOps e os fundamentos da integração contínua, explorando problemas, soluções e a ferramenta Jenkins, com ênfase na relação com a gestão de liberação.

Rodrigo Augusto dos Santos

1. Itens iniciais

Propósito

A compreensão de DevOps e integração contínua é fundamental para profissionais de TI que atuam com gerência de configuração, pois permite automatizar processos, reduzir erros e acelerar entregas, contribuindo diretamente para a eficiência e qualidade dos sistemas em ambientes de desenvolvimento e produção.

Objetivos

- Analisar os fundamentos, as boas práticas e a aplicação da Integração Contínua com apoio da ferramenta Jenkins, destacando suas vantagens e limitações no processo de desenvolvimento de software.

- Distinguir os principais elementos conceituais, culturais e operacionais do DevOps, reconhecendo os papéis envolvidos e os impactos positivos e negativos dessa abordagem na entrega contínua de software.

Introdução

Neste conteúdo, será abordado um dos tópicos mais atuais e relevantes dentro da gerência de configuração: a integração contínua. Esse conceito tem ganhado cada vez mais destaque nas práticas de desenvolvimento de software por permitir maior agilidade, confiabilidade e automação nos processos de entrega. A integração contínua promove a integração frequente de códigos desenvolvidos por diferentes membros de uma equipe, o que reduz significativamente os conflitos de versão e facilita a detecção de erros ainda nas etapas iniciais do desenvolvimento.

Ao longo do material, serão apresentados alguns dos principais problemas enfrentados durante a implementação da integração contínua, bem como soluções práticas e eficazes para superá-los. Para isso, será dada ênfase especial à utilização do Jenkins, uma das ferramentas mais populares e robustas para automação de pipelines de integração e entrega contínuas. A escolha do Jenkins como foco se deve à sua ampla adoção no mercado e à versatilidade que oferece para diferentes contextos e projetos.

Além disso, será explorado o conceito de DevOps, uma abordagem que vai além da técnica e propõe uma mudança cultural na forma como times de desenvolvimento e operações colaboram. O DevOps está no centro da gerência de configuração moderna e estabelece uma base sólida para a entrega contínua de software com qualidade e velocidade.

Com este estudo, você terá a oportunidade de entender como esses conceitos se conectam e contribuem para otimizar processos, aumentar a produtividade das equipes e entregar soluções mais estáveis e eficientes aos usuários finais.

A integração contínua e seus fundamentos

Vimos que o gerenciamento de configuração, por meio de práticas componentes diversas, tais como controle de mudança e gerenciamento de liberação, tornou-se fundamental para projetos de desenvolvimento de software, bem como para sua manutenção e evolução.

Comentário

Tal importância fez do gerenciamento de configuração algo muito bem estabelecido dentro da comunidade de software mundial, sendo basicamente adotado, mesmo que parcialmente, por praticamente qualquer organização moderna. O gerenciamento de configuração pode ser considerado uma commodity, algo “tradicional” e que, por si só, não é mais fonte de vantagem competitiva.

Com a ascensão e, até certo ponto, consolidação dos frameworks ágeis nas décadas de 2000 e 2010, uma nova cultura voltada principalmente a projetos passou a existir, dando muito mais enfoque à flexibilidade e prontidão das equipes de desenvolvimento.

Novas práticas, agora mais ágeis, passaram a ser adotadas por essas equipes. Junte-se a isso a própria evolução das várias tecnologias empregadas no desenvolvimento de software, cada vez mais complexas e integradas, que exigiram times mais diversos e multidisciplinares.

> Todos esses fatores afetaram o gerenciamento de configuração, que evoluiu para, entre outras coisas, incluir também a prática conhecida como integração contínua. Tal prática foi originalmente proposta em 1991, ou seja, antes do manifesto ágil e, principalmente, antes dos frameworks ágeis ganharem popularidade.

É com a popularização destes *frameworks*, no entanto, que a integração contínua encontra mais do que nunca terreno fértil para justificar sua ampla adoção.

Exemplo

Um exemplo disso é o framework ágil conhecido como eXtreme Programming (XP), em que a prática da integração contínua passou a ter o status de princípio a ser seguido, junto a vários outros princípios reconhecidamente fundamentais para o XP, tais como os que envolvem o nível adequado de envolvimento e participação nos projetos por parte dos clientes, programação em pares promovendo a responsabilidade e propriedade coletiva sobre o código-fonte etc.

O XP teve papel predominante na difusão da integração contínua.

A integração contínua começa por endereçar o problema fundamental da integração de todo o código-fonte criado por uma equipe composta por diversos desenvolvedores de software. Nos anos 1990 e início dos anos

2000, a integração de múltiplas linhas de código, envolvendo checagem e teste, levava semanas ou mesmo meses de trabalho.

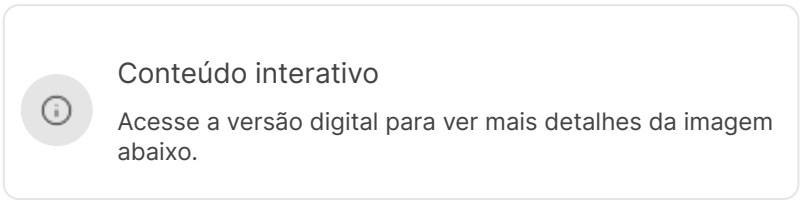
Obviamente, tal abordagem propiciava problemas na integração, tais como muitos defeitos e diferenças que precisavam ser retrabalhadas (nomes de variáveis diferentes, parâmetros faltantes etc.).

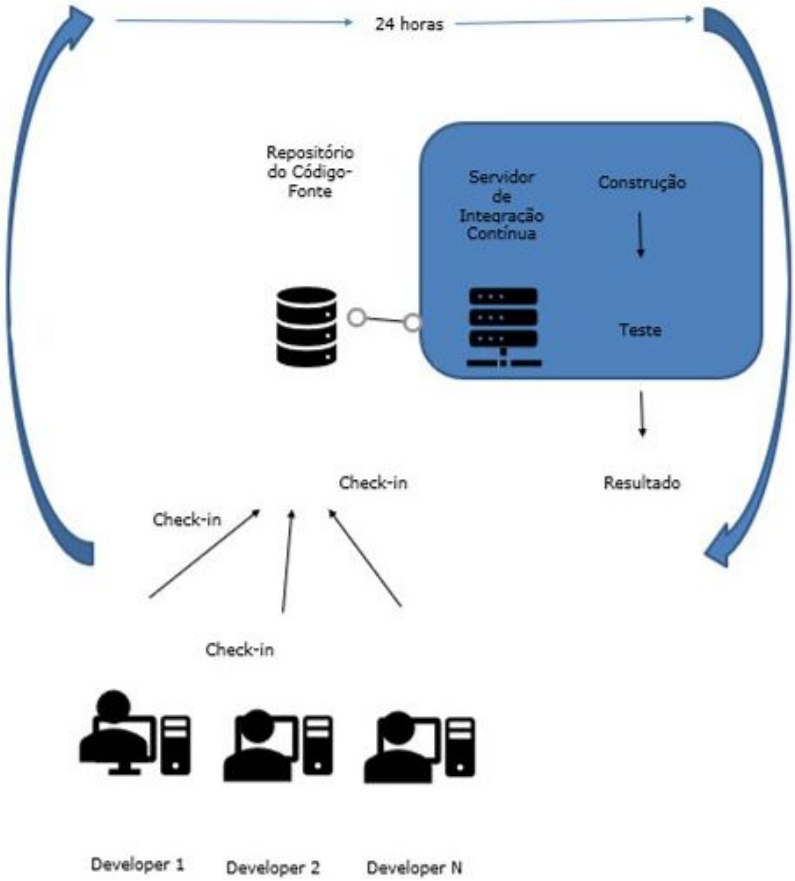
Tais problemas foram minimizados com a adoção da integração contínua, que em seu cerne se trata de continuamente fundir ou unificar atualizações no código-fonte por parte de todos os desenvolvedores da equipe, em uma versão do código considerada a versão principal, compartilhada e visível a todos.

A principal ideia é assegurar que nenhum desenvolvedor em particular vá se desviar demais da linha de trabalho geral adotada pela equipe, constantemente garantindo-se que o código-fonte esteja em estado de integração, evitando-se falhas catastróficas de integração e o consequente retrabalho.

A integração contínua evoluiu a um ponto em que, nos dias de hoje, ela deve ocorrer no mínimo uma vez por dia, ou seja, pelo menos uma vez diariamente deve-se garantir que todo o código-fonte produzido por todos os desenvolvedores naquele dia em específico irá integrar-se, ou seja, será construído e testado automaticamente.

Esse ciclo de integração ocorre em um servidor de integração contínua, em que os pontos a seguir normalmente ocorrem em sua totalidade.





Fluxo básico da integração contínua.

Detectar mudanças nos itens de configuração

No caso da integração contínua, os itens de configuração serão os diversos códigos-fonte. Eles se encontrarão armazenados no repositório de alguma ferramenta de controle de versão, tal como SVN ou Git.

Mover mudanças para o servidor

As mudanças detectadas nos itens de configuração são migradas do repositório para o servidor de integração contínua.

Compilar as mudanças no código-fonte

Aqui o código é de fato integrado por meio da compilação de cada item de configuração em conjunto com os demais, gerando os executáveis e outros arquivos de apoio necessários para executar o software.

Realizar testes de qualidade automatizados

Tais como testes unitários, de arquitetura, de design etc. Análise de qualidade do código também pode ocorrer. Esses testes em geral são opcionais, mas altamente recomendados para assegurar a geração de vantagens a que se propõe a integração contínua.

Inserir código compilado no repositório

Dado que o código compile e integre sem erros, ele é inserido (*checked-in*) novamente no repositório de onde os itens de configuração originais foram retirados no primeiro passo.

Reportar o status da construção às partes interessadas

Normalmente por meio de algum mecanismo automatizado de e-mail, SMS etc.

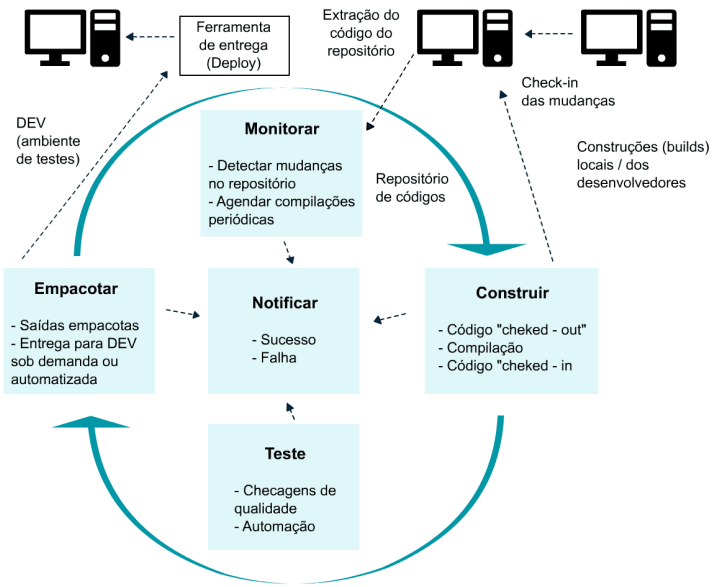
Empacotar e entregar o código compilado para testes

Outro passo opcional, se trata de entregar o novo código no ambiente de testes apropriado.

Vejamos agora um exemplo prático de como a integração contínua se comporta:

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Passos detalhados da integração contínua.

Um desenvolvedor tem uma tarefa a executar em relação a um pedaço de código-fonte (adicionar alguma funcionalidade, corrigir algum erro). Ele obtém (check-out) uma cópia do código-fonte atualizado e integrado disponibilizada em sua estação de trabalho. Tal obtenção é feita por meio da ferramenta de controle de versão do código-fonte adotada. Nesse exemplo, faremos uso hipotético do SVN.

O desenvolvedor faz o que precisa fazer e termina sua tarefa, o que inclui não só realizar as mudanças, mas também adicionar ou alterar testes automatizados necessários. Ele então realiza a construção (build), pega o código fonte alterado, compila ligando a um executável, e roda localmente os testes automatizados. Se a construção e os testes ocorrerem sem erro, então a construção (build) é boa.

O desenvolvedor então confirma (commit) as mudanças no repositório. O problema disso é que outros desenvolvedores podem ter feito suas próprias mudanças no código-fonte base antes mesmo que o desenvolvedor tenha tido chance de realizar o commit. Assim sendo, ele precisa atualizar seu código-fonte alterado com as mudanças realizadas pelos outros desenvolvedores e reconstruir.

Se alguma mudança de terceiros conflitar com suas próprias mudanças, isso irá gerar erros na compilação ou nos testes. É responsabilidade do desenvolvedor corrigir os problemas e repetir a construção até que tudo funcione de forma sincronizada com a linha principal do código.

Uma vez que isso seja atingido, ele pode finalmente realizar o commit das mudanças na linha principal do código, o que atualiza o repositório. O trabalho ainda não está encerrado. Dessa vez é preciso construir novamente, só que no servidor de integração contínua. Apenas após isso ocorrer com sucesso é que as mudanças do desenvolvedor podem ser consideradas completas.

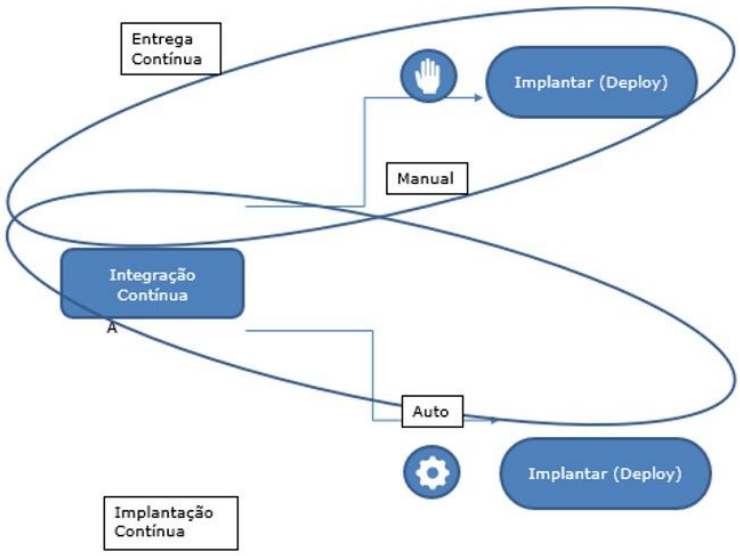
Isso ocorre porque sempre existe a chance de que o desenvolvedor tenha esquecido alguma coisa ou mesmo que o repositório não tenha sido totalmente atualizado corretamente.

Independentemente de quem encontre erros (se o desenvolvedor original, ou algum outro desenvolvedor), o ponto é que falhas serão descobertas rapidamente, sendo que elas não devem ser deixadas de lado, e sim ser consertadas o quanto antes.

Por fim, a integração contínua não deve ser confundida com entrega contínua, que é a manutenção de uma aplicação sempre em um estado pronto para ser implantada (deployed) manualmente. Já a implantação contínua automatiza esse passo, ou seja, é automação da construção, teste e implantação.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Integração contínua e suas peculiaridades.

Assista o vídeo e saiba mais sobre gestão de configuração.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

As vantagens e desvantagens da integração contínua

A integração contínua é amplamente adotada simplesmente porque traz vários benefícios tangíveis à construção de software. Porém, como nem tudo é vantagem, listamos na tabela a seguir algumas vantagens e desvantagens ou problemas da integração contínua.

Vantagens

- **Mudanças menores de código**: A integração contínua permite a integração de pequenas partes de código-fonte a cada vez. Essas mudanças menores são mais simples de lidar, ao contrário de grandes pedaços de funcionalidade, e, portanto, geram menos problemas. O uso de teste (contínuo) permite que os desenvolvedores descubram problemas antes de grande parte do trabalho ter sido realizada, funcionando bem para times pequenos e grandes, colocados ou remotos.

- **Isolamento de falhas**: Refere-se à contenção de impactos, ou seja, projetar os sistemas para que, quando erros ocorram, os impactos negativos sejam limitados em escopo, reduzindo danos e fazendo com que os sistemas sejam mais fáceis de serem mantidos. O Isolamento de falhas combina monitoração do sistema, identificando quando a falha ocorre e apontando sua localização, o que ajuda a isolar e possivelmente desativar a funcionalidade afetada antes que ela cause danos ao sistema inteiro.

- **Mais confiabilidade nos testes**: Usar integração contínua faz com que a confiabilidade dos testes melhore, dado o caráter de automação e previsibilidade introduzido. Tudo é mais repetível.

- **Liberações mais rápidas**: Por automatizar testes e integração, por si só a integração contínua ajuda a gerar liberações mais rápidas. Se o escopo for expandido para incluir capacidades de entrega contínua ou implantação contínua, os ganhos em velocidade serão ainda maiores. A exigência aqui é ao menos manter o código em estado pronto para implantação a qualquer momento. Para isso, é importante prestar muita atenção na manutenção de ambientes de desenvolvimento e testes os mais próximos possíveis ao ambiente de produção.

- **Satisfação do cliente**: A rápida entrega de novas funcionalidades e correções de defeitos tende a gerar a imediata satisfação dos clientes. Outra vantagem da integração contínua e das rápidas entregas é que propiciam um ambiente mais aberto à adoção de tecnologias novas e mais recentes. Esses pontos ajudam com clientes atuais, mas também ajudam a ganhar novos clientes.

- **Redução de custos**: Já que traz foco para automação, a integração contínua reduz o uso de desenvolvedores caros em tarefas corriqueiras e repetitivas. Também reduz o número de erros e de custos com retrabalho, liberando os desenvolvedores para trabalharem realmente no desenvolvimento de produtos.

Desvantagens

- **Necessidade por mudança cultural**: Algumas organizações ainda preferem trabalhar de forma tradicional, e podem apresentar muita resistência quando o tema é a implementação de integração contínua. Elas sabem que precisam retreinar funcionários, o que ainda implica em reformar as operações existentes. Gerentes podem ter outras prioridades imediatas e podem, portanto, perceber a integração contínua como mais uma distração.

- **O custo da transição**: Além de todo o óbvio esforço de adoção, existem ainda investimentos consideráveis em forma de tempo e custos. Alterar fluxos de trabalho, automatizar o processo de teste, migrar Código para dentro de repositórios, todos estes são itens que precisarão ser contemplados. Todos são viáveis de serem atingidos, mas não serão baratos. Em especial para o caso de equipes que lidam sempre com projetos muito pequenos, esse custo pode ser inaceitável.

- **Dificuldade em manter**: Construir um repositório automatizado de código não é fácil. Equipes precisam construir uma suíte de teste e gastar tempo escrevendo casos de teste ao invés de desenvolver código. Em um primeiro momento, isso irá torná-los mais lentos e talvez dificulte terminar projetos nos prazos previamente acordados. Isso sem considerarmos o retrabalho que essas atividades em algum momento exigirão, pois é impossível fazer tudo perfeitamente já no primeiro momento.

- **Muitas mensagens de erro**: Ocorrem principalmente quando os times de desenvolvimento são grandes; há tantas mensagens de erro nos itens de configuração que os profissionais passarão a ignorá-las, já que eles têm outras coisas a fazer. Isso pode levar a construções quebradas, que vão cada vez mais se acumular se não forem cuidadas a tempo.

- **Falta de pessoal capacitado**: Como existe muita automação de testes envolvida, pode ser que a organização não disponha de pessoal suficiente com conhecimento em testes, obrigando-a a fazer uso de desenvolvedores nestas funções. Alguns testes são notoriamente difíceis de automatizar; logo, se o sistema a ser testado dispõe de módulos complexos, com muitos elementos visuais, a escrita dos testes pode se tornar muito desafiadora e consumir tempo demais.

Assista ao vídeo e saiba mai sobre a integração contínua.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Boas práticas na integração contínua

A integração contínua, por abranger no dia a dia muitos passos e profissionais, precisa ser regida por uma série de boas práticas que foram sendo compiladas e refinadas ao longo do tempo em vários campos de aplicação. Listamos a seguir algumas das práticas mais amplamente aceitas e difundidas, algumas das quais foram adaptadas do autor Martin Fowler.

Automatize a construção (*Build*)

Significa tornar as fontes de código em um sistema que possa ser executado — e de maneira automática.

Com as ferramentas disponíveis atualmente, ninguém precisa fazer isso manualmente, ou seja, digitar múltiplos comandos e clicar em várias caixas de diálogo, o que invariavelmente aumenta a chance de cometer erros. Essa abordagem sujeita a erros deve ser substituída por um único comando capaz de construir o sistema inteiro.

Torne a construção em auto-testável

Assim que o código estiver construído, o máximo possível de testes automatizados deve rodar para confirmar que ele se comporta conforme expectativa dos desenvolvedores.

Isso é necessário, conforme já vimos anteriormente, para que se aumente a eficiência e eficácia na identificação de defeitos. Além da economia de tempo, é importante mencionar que, sem contemplar esse ponto, ao menos parcialmente, não é possível ter Integração Contínua.

Mais uma vez, o ideal é que os testes que foram automatizados sejam executados por meio de um único comando e sejam abrangentes em termos de cobertura do código-fonte. Por fim, testes automatizados não substituem completamente testes manuais, então é importante manter isso em perspectiva.

Mantenha a construção rápida

A construção deve ser concluída rapidamente, de forma que, se houver problemas com integração, eles sejam identificados rapidamente. Para se ter uma ideia, o *eXTreme Programming* mais purista prega *builds* de 10 minutos.

Torne fácil a obtenção dos entregáveis mais recentes

A obtenção de construções por parte das partes interessadas e testadores pode reduzir a quantidade de retrabalho necessário quando se precisa reconstruir alguma funcionalidade que não atinge os requisitos.

O teste precoce reduz as chances de que defeitos chegarão até a implantação. Encontrar erros o mais cedo possível também reduz custos e a quantidade de trabalho necessário para consertá-los.

Entenda as construções mais recentes

Deve ser fácil descobrir se uma construção apresentou problemas e quais mudanças relevantes fizeram parte dela.

Automatize as implantações

Todo o processo deve ser automatizado por meio de scripts inseridos no sistema de integração contínua.

Isso é ainda mais verdadeiro se considerarmos que muitos ambientes são requeridos pela integração contínua, além do fato de que executáveis são movimentados entre esses ambientes de forma frequente, possivelmente múltiplas vezes ao dia.

Em adição à implantação automatizada, é importante possuir também remediação automatizada, ou seja, a capacidade de desfazer as implantações caso problemas surjam.

Todos confirmam (*commit*) na linha mestre de código todos os dias

Esse é o aspecto mais visível e entendido da integração contínua, chave ao redor da qual a Integração Contínua está estruturada.

Quanto mais frequente esse passo for, melhor será a integração, pois menos erros resultantes de conflitos persistirão, já que serão rapidamente encontrados e tratados.

Toda confirmação (*commit*) deve construir a linha mestre de código

Toda confirmação (*commit*) deve construir a linha mestre de código em uma máquina voltada para a integração

Ao usar uma máquina de integração, os desenvolvedores se certificam de que suas construções foram bem sucedidas e que estão realmente prontas para a linha mestre.

Isso pode ser feito manualmente ou por meio de um servidor de integração contínua. Obviamente, a maneira automatizada é a preferível, sendo a alternativa na qual daremos foco.

Conserte construções quebradas imediatamente

Se a linha mestre falhar, deve então ser consertada imediatamente. A ideia é fomentar uma base sempre o mais estável possível. É natural que a linha mestre quebre de tempos em tempos, mas não o tempo todo.

Se quebras constantes ocorrem, provavelmente significa que os desenvolvedores não estão finalizando as construções localmente antes de realizarem a confirmação (*commit*) na linha mestre.

A todo momento, o conserto da linha mestre deve ser prioridade para todos os membros do time.

Uma forma de fazer o conserto é reverter para o *commit* anterior da linha mestre e então realizar o debug do problema em uma estação de trabalho.

Use clones do ambiente de produção

Os testes, em especial os automatizados, usados na integração contínua funcionam melhor se conduzidos em ambientes que são clones virtuais do ambiente de produção.

Obviamente, se os ambientes forem diferentes os resultados serão diferentes; portanto, é interessante que o espelhamento de ambientes inclua, mas não se limite a, bancos de dados, sistemas operacionais, bibliotecas, endereços de IP e portas, hardware etc.

Torne fácil a obtenção do último executável por qualquer pessoa

Uma das razões pelas quais a integração contínua foi idealizada é que antes se levava muito tempo para construir o software certo. A integração contínua reconhece que muitas vezes se percebe que algo não está certo, e que, portanto, precisa mudar.

Apesar do desafio em lidar com múltiplas mudanças de múltiplas fontes ao mesmo tempo, a Integração Contínua exige que qualquer desenvolvedor seja capaz de rodar o mais recente executável para demonstrações, testes exploratórios ou qualquer coisa.

Todos devem ser capazes de ver o que está acontecendo

Uma boa integração contínua faz uso consistente de boa comunicação, não somente entre os desenvolvedores e outros perfis trabalhando na construção de software, mas também entre eles e os donos do produto, permitindo que todos conheçam o estado do sistema e as mudanças realizadas.

Em outras palavras, todos têm conhecimento do que a linha mestre do software inclui/representa. A maioria do software de suporte à Integração Contínua mostra o progresso das construções e o estado atual da linha mestre, o que é útil em prover a visibilidade esperada.

Apesar disso, outras formas de transmitir essa informação existem, tais como uso de código de cores (verde para sucesso, vermelho para falha), avisos sonoros etc.

Assista o vídeo e conheça algumas boas práticas sobre integração contínua.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

A ferramenta Jenkins e seu uso voltado para integração contínua

Já falamos anteriormente da ferramenta Jenkins, a conhecida ferramenta *opensource*, sendo que focamos em suas capacidades de gestão de construção. Nessa aula, daremos um enfoque diferente, concentrando em sua capacidade para a integração contínua.

A ferramenta na verdade se tornou popular exatamente por causa dessa capacidade, já que ela é capaz de acelerar todo o processo de desenvolvimento de software por meio da abordagem integrada entre a automação de construção e a automação de testes, tudo de maneira rápida e simples.

Vale mencionar que a ferramenta pode suportar ainda a documentação, implantação e mais estágios do ciclo de vida do software.

Saiba mais

Para isso, precisamos entender o conceito de Tubulação de Integração Contínua (Pipeline de Integração Contínua), que nada mais é do que um grupo de ferramentas integradas trabalhando em conjunto para hospedar, monitorar, compilar e testar código-fonte e/ou mudanças em código.

O Jenkins é um dos elementos do *pipeline*, já que faz o papel de Servidor de Integração Contínua. Os outros elementos mínimos para compor o *pipeline* são as ferramentas de:

- Controle de Versão: Para esse papel, vimos as ferramentas SVN e Git;
- Ferramenta de Construção: Vimos as ferramentas Maven e Gradle;
- Ferramenta de Automação de Testes: Várias existem, sendo que uma das mais conhecidas é a Selenium.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

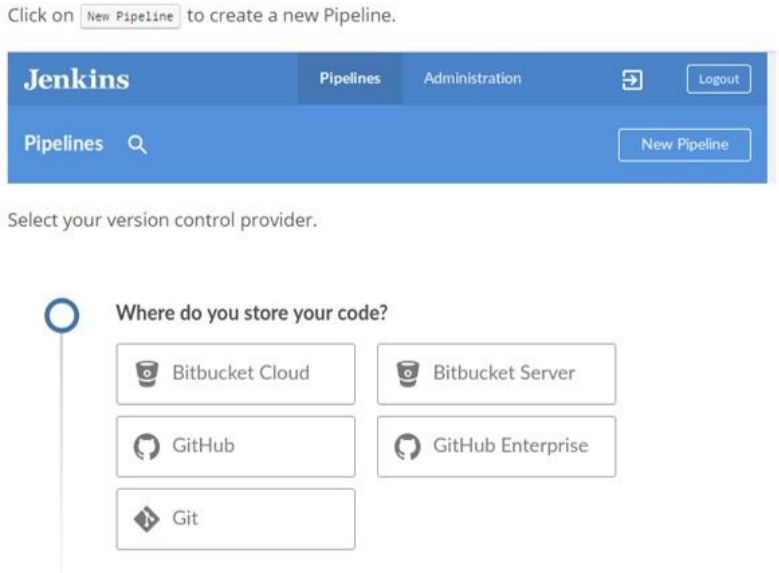


Exemplo de construção de pipeline com dois estágios.

A construção dos *pipelines* é muito simples, podendo ser via código em arquivo de configuração (*JenkinsFile*) ou via interface gráfica, já que o Jenkins apresenta boa flexibilidade em trabalhar com diversos plugins. Citaremos o plugin Blue Ocean para a configuração do pipeline, bem como os diversos plugins isolados das ferramentas com as quais se deseja integrar e fazer uso; por exemplo, os plugins do Git, Gradle, Maven etc.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Exemplo de construção de pipeline com via interface gráfica.

Fica claro que o Jenkins é mais do que uma ferramenta, sendo na verdade um framework voltado para facilitar atividades de desenvolvimento de software por meio de integração e orquestração de uma série de ações. O Jenkins permite, portanto, a plena adoção da integração contínua, bem como que as organizações e os times de desenvolvimento vão além, adotando a entrega contínua e/ou a implantação continua.

Verificando o aprendizado

Questão 1

A integração contínua é fornecida pela junção flexível e customizável das funcionalidades proporcionadas por diversas ferramentas, cada uma trabalhando em seu nicho específico. As organizações adotam o que melhor se encaixa nas suas estruturas de trabalho. Nesse sentido, podemos considerar a integração contínua como:

A Um script.

B Um aplicativo.

C Um método.

D Um framework.

E Um roteiro.

A alternativa D está correta.

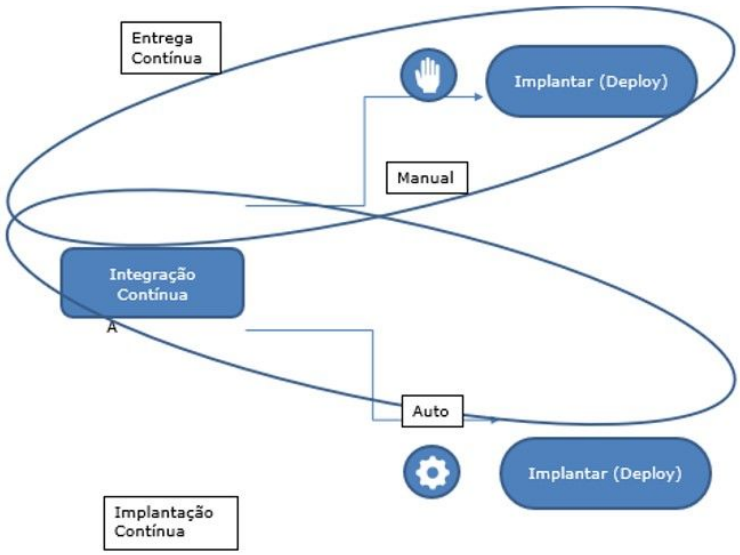
Como as organizações são livres para adotar o que faz sentido para suas condições, pode-se dizer que a integração contínua é um framework, ou seja, uma descrição do que precisa ser feito sem adentrar nos detalhes de como deve ser feito. Esse framework será composto por decisões, pela escolha de práticas, scripts e ferramentas que entreguem o que é preciso.

Questão 2

A integração contínua, embora seja uma prática poderosa, não é a prática mais completa possível. Nesse sentido, temos práticas que podem ser consideradas como extensões da integração contínua. No diagrama a seguir, mostramos duas delas. A qual prática a elipse verde se refere?

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



A Implantação contínua.

B Entrega contínua.

C Liberação em pacote.

D Liberação emergencial.

E Liberação maior.

A alternativa B está correta.

A entrega contínua significa manter o software sempre em estado pronto para ser implantado. Apesar disso, a implantação em si ocorre de forma manual, sem automatização significativa.

Questão 3

No contexto da integração contínua, isolar falhas se refere a:

A Servidores redundantes.

B *Commits* locais antes do servidor de IC.

C *Commits* locais antes da linha mestre de código.

D Completa.

E Desenho para limitação do escopo de erros.

A alternativa E está correta.

O isolamento de falhas combina monitoração do sistema, identificando quando a falha ocorre e apontando sua localização. Isso ajuda a isolar e possivelmente desativar a funcionalidade afetada antes que ela cause danos ao sistema inteiro. É necessário o desenho específico do software para que essa capacidade seja atendida.

Questão 4

Qual dos itens abaixo é uma boa prática pregada pela integração contínua?

A Manter a construção rápida.

B Definir o que a organização quer obter com o gerenciamento de configuração.

C Tratar *commits* de forma atômica.

D Usar ativamente os metadados.

E Instituir um gerente de liberações.

A alternativa A está correta.

A ideia é identificar erros de construção e integração o mais rapidamente possível, de forma que também possam ser rapidamente corrigidos. Por exemplo, o framework ágil eXTreme Programming, fortemente inclinado à integração contínua, prega builds de 10 minutos.

Questão 5

A ferramenta Jenkins faz principalmente o papel de:

A Ferramenta de controle de versão.

B Servidor de integração contínua.

C Ferramenta de construção.

D Ferramenta de liberação.

E Ferramenta de teste.

A alternativa B está correta.

A ferramenta Jenkins representa o servidor de integração contínua, se integrando a outras ferramentas necessárias para entregarem o pipeline de integração contínua completo.

O DevOps e seus fundamentos

Ao discutirmos previamente o conceito de integração contínua, mencionamos brevemente o contexto ágil em que tal conceito se encontrava inserido. Basicamente, usamos a tendência cada vez maior do uso de frameworks ágeis, tais como o *eXtreme Programming* (XP), para justificar, até certo ponto, a adoção da integração contínua.



Comentário

É importante reforçar o caráter parcial que demos a esta justificativa, pois a agilidade não era e não é o único fator que justifica a existência da Integração Contínua como prática aliada à gerência de configuração.

Ao falarmos de DevOps, um conceito mais avançado do que integração contínua, e que também pode ser associado à gerência de configuração, precisamos novamente voltar à tendência da agilidade, pois diferentemente da integração contínua, no caso do DevOps, esta tendência se encontra bem no cerne de sua adoção.

O termo em si surgiu pela primeira vez em 2008, após o manifesto ágil e dentro do período em que *frameworks* ágeis, tais como XP, Scrum, Lean e outros, se expandiram e popularizaram, influenciando profissionais e organizações como um todo. Não surpreende que o DevOps já tenha sido conhecido por "infraestrutura ágil" ou "operações ágeis".

> O DevOps é a junção entre Desenvolvimento (Development) e Operações (Ops). É uma mudança de paradigma fundamental em relação às abordagens tradicionais, em que a fase do desenvolvimento, ou seja, a fase de projetos, sempre foi encarada como sendo completamente segregada da fase de operações.

Esta conceituação do DevOps deixa clara a presença de uma mentalidade ágil, pois com a junção de ambas as fases busca-se obter um relacionamento agilizado entre elas, melhorando a comunicação e encurtando-se o tempo de desenvolvimento, ao mesmo tempo em que produz software de melhor qualidade.

O DevOps facilita a comunicação e a colaboração através das organizações, objetivando a produção e melhoria de produtos de software em um ritmo maior do que seria possível caso abordagens tradicionais fossem utilizadas.

Além disso, de maneira similar (e por que não complementar?) à integração contínua, dedica forte atenção em automação de tarefas, de forma a liberar os desenvolvedores de software para que foquem no que realmente entrega valor: Desenvolvimento de código e outras tarefas que, em larga medida, não podem ser automatizadas; ao menos, não em sua totalidade.

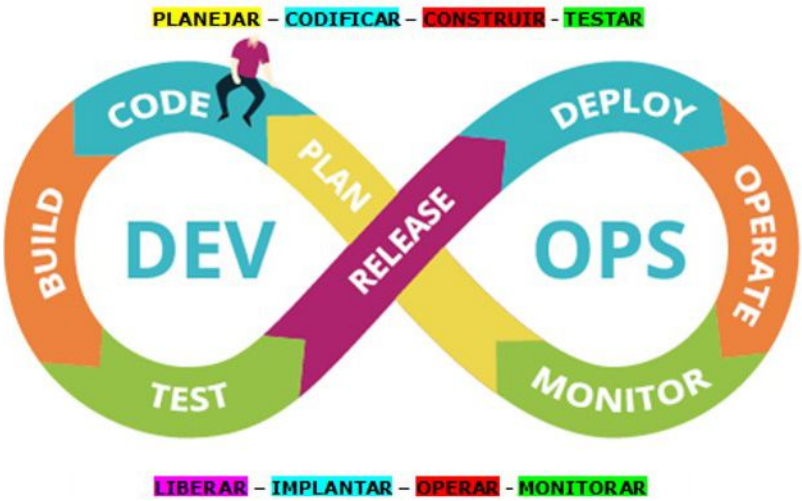
Saiba mais

Esta automação entre desenvolvimento e operação entrega construção, testes e liberação, além de contar com o uso extensivo de filosofia de trabalho, práticas e habilidades próprias. Entre as várias tecnologias usadas para se atingir este fim, podemos citar por exemplo máquinas virtuais e de containerização (containers).

Entre as práticas, podemos citar, por exemplo, a participação conjunta entre engenheiros de desenvolvimento e de operação no ciclo de vida inteiro do produto, dos requisitos ao suporte continuado.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



O ciclo integrado do desenvolvimento e operação.

O DevOps se organiza ao redor de três princípios:

O princípio do fluxo

A ideia aqui é reduzir o tempo da confirmação (*commit*) ao código rodando em produção.

O princípio do feedback

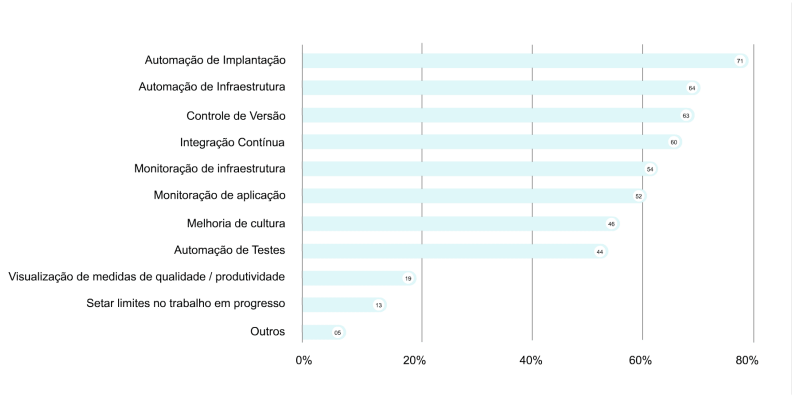
Deve-se aumentar o *feedback* do desenvolvimento para produção e da produção para o desenvolvimento.

O princípio do aprendizado contínuo e experimentação
Aqui se reconhece a importância de se aprender continuamente e usar este aprendizado para melhorar os processos envolvidos.

O DevOps se estrutura ainda ao redor de várias práticas fundamentais, que podem ser adotadas de forma isolada ou em conjunto. De fato, é comum que as organizações se encontrem em níveis diferentes de maturidade na adoção do DevOps, conforme indicado pelo gráfico extraído do Survey Global em DevOps, conduzido no ano de 2017. Note que algumas práticas ainda são muito pouco adotadas.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

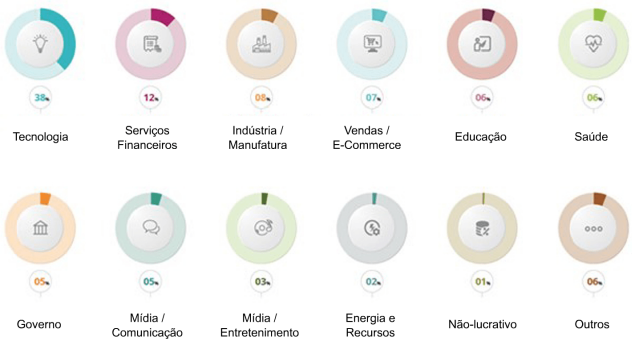


Adoção de práticas do DevOps.

Outro dado que demonstra claramente que a adoção do DevOps como um todo ainda se encontra em andamento, devendo evoluir muito nos próximos anos, é o gráfico de adoção do DevOps por segmento. Não é surpreendente que a indústria da tecnologia, mais especificamente a Indústria de tecnologia da informação, esteja na vanguarda da adoção, com ampla vantagem sobre o segundo colocado, no caso a indústria de serviços financeiros.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Adoção do DevOps por segmento.

Assista o vídeo e saiba mais sobre o DevOps.

> **Conteúdo interativo**
>
> Acesse a versão digital para assistir ao vídeo.

A cultura que suporta a adoção e manutenção do DevOps

O principal ponto cultural que precisa mudar ao se decidir pela adoção do DevOps é a eliminação dos chamados silos funcionais, ou seja, a concepção de que equipes totalmente diferentes devem existir para desenvolvimento e operação de software.

É possível que duas equipes formais separadas ainda existam para cumprir estes propósitos; contudo, no DevOps a linha que as separa se torna cada vez mais difusa, até um ponto em que se perceba que é melhor fundir as duas equipes em uma única entidade.

> **Comentário**
>
> Independentemente da abordagem seguida (se separar ou fundir), o DevOps extingue a cultura da "passagem do bastão", ou seja, a cultura que se centrava em um ponto de corte, em que o software, recém entregue pela equipe de projetos, era repassado à equipe de operações, que passaria então a ser a responsável por suportar o software adiante.

Esta cultura anterior era baseada mais em performances individuais, tanto das equipes em si quanto dos profissionais que as integravam.

No DevOps, ao contrário, se valoriza o compartilhamento de responsabilidades e a flexibilidade em se ter profissionais trabalhando em pontos ou atividades distintas, sendo que o valor é gerado ao se atender a necessidade imediata do esforço de desenvolvimento ou operação corrente.

> **Exemplo**
>
> Se um desenvolvedor está disponível e existe a necessidade em se corrigir defeitos vindos da operação de um produto de software, então não deve ser problema que tal desenvolvedor possa realizar a correção ou mesmo testar a correção.

Outro ponto importante derivado dessa mudança de mentalidade é o endereçamento de um problema que afeta organizações de TI ao redor do mundo: A taxa de rotatividade (turnover). Não é incomum encontrar organizações enfrentando problemas com membros-chave, deixando as equipes, seja de projetos, seja de operações, também em momentos chave ou críticos.

Com a nova cultura de compartilhamento de responsabilidades, é possível expor membros do time a vários tipos de trabalho.

Isso leva ao aumento do conhecimento em relação ao todo, fazendo com que não só contribuam em algum momento com todo o ciclo de desenvolvimento, implantação e sustentação, mas também reduzindo a dependência em relação a profissionais específicos, tornando todo a organização mais segura, e também protegendo a relação com o cliente, que normalmente é quem vê diretamente o impacto negativo da rotatividade.

Dados estes fatores, fica claro que a adoção do DevOps não ocorre com sucesso sem uma estratégia clara para se quebrar os silos funcionais.

A integração contínua e, mais especificamente, sua evolução encontrada na entrega contínua também são pilares do DevOps, e, portanto, devem ter sido internalizadas culturalmente pelas organizações antes que o DevOps possa vir a ser adotado.

Tais práticas automatizam partes importantes do desenvolvimento e implantação, liberando as equipes para focarem em pontos mais críticos, como, por exemplo, em reduzir os tempos de desenvolvimento e a apresentarem liberações mais rápidas, frequentes e com menos erros. Um panorama da automação requerida pelo DevOps pode ser vista a seguir.

Construir uma cadeia de ferramentas aderentes

- Continuamente revisar as cadeias de ferramenta e melhorá-las conforme necessário.
- Delegar um dono para as ferramentas.
- Automatize a governança de ferramentas para software Open Source com relação a segurança e controle de licenças.

Colaborar e simplificar

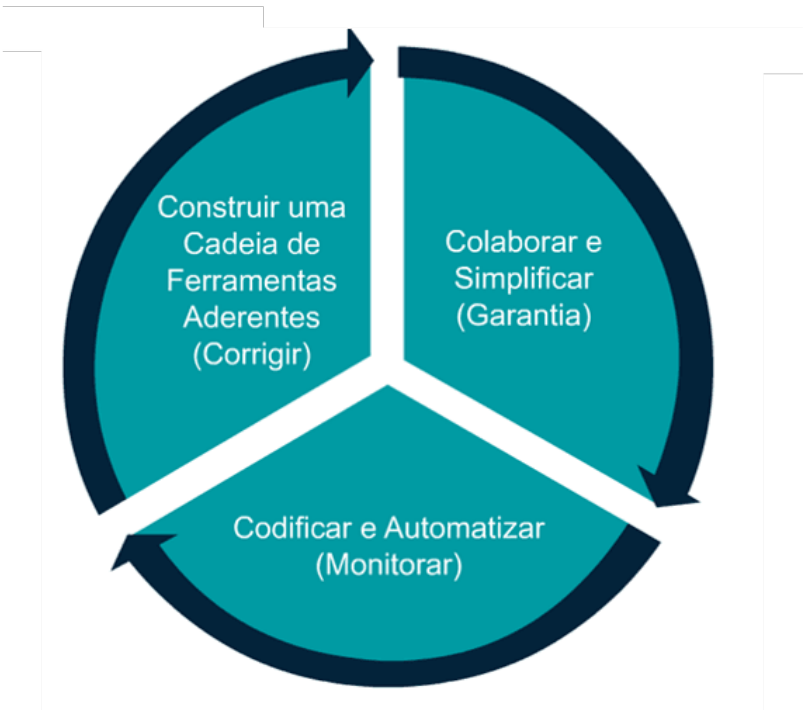
- Colaborar com todas as partes interessadas.
- Simplificar controles e eliminar aqueles desnecessários.
- Reusar controles compartilhados.
- Documentar.
- Buscar aprovações.

Codificar e automatizar

- Codificar controles.
- Codificar fluxos de trabalho.
- Integrar controles à cadeia de ferramentas.
- Adquirir ferramentas de compliance.
- Continuamente monitorar, reportar e notificar.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Passos para adoção e compliance com DevOps.

A existência de automação no estado da arte ajuda ainda as equipes técnicas a focar nos clientes e suas necessidades, uma clara distinção que separa o DevOps de abordagens mais tradicionais.

Aliás, o DevOps estimula a comunicação próxima entre desenvolvedores e clientes de forma a assegurar que o software desenvolvido atinja as expectativas iniciais, além é claro de trazer a vantagem secundária de que problemas sejam endereçados mais rapidamente do que era possível anteriormente.

Dada a natureza das mudanças organizacionais necessárias, nem toda organização está preparada para adotar o DevOps, seja por falta de apoio, seja por considerar outras prioridades. Outro ponto que pode ser um inibidor inicial é o receio de que a agilidade não combine com regulações, por exemplo. Apesar da preocupação em si ser válida, é possível aliar as duas perspectivas.

A conhecida empresa de consultoria Gartner propõe três passos para a adoção inicial, bem como adoção continua do DevOps, sempre em alinhamento com requisitos regulatórios. Tais passos podem ser vistos na figura a seguir.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Possíveis aspectos a serem cobertos por ferramentas em um ambiente DevOps.

Cases de sucesso

Vejamos agora alguns casos de sucesso na adoção do DevOps.

Enquanto o Walmart é o rei das empresas de venda baseadas em lojas físicas nos Estados Unidos, no mundo online a empresa sempre esteve às sombras da Amazon. Visando ganhar espaço, o Walmart montou um time avançado por meio de várias aquisições, fundando o WalmartLabs, que nada mais é do que o braço tecnológico, de inovação e desenvolvimento da empresa.

Uma das abordagens chave da missão do WalmartLabs é o uso do DevOps. Ele incorporou a tecnologia OneOps baseada na nuvem, que automatiza e acelera a implantação de aplicações. O WalmartLabs também criou várias ferramentas abertas, como o Hapi, um *framework* baseado em Node.js para construção de aplicações e serviços que permitem aos desenvolvedores focarem em escrever lógica reutilizável de aplicações ao invés de gastarem tempo com infraestrutura. Mais recentemente, implantou mais de 100,000 cores OpenStack de forma a construir e dispor de sua própria nuvem privada, continuando a evoluir sua abordagem ágil.

Quando a Netflix evoluiu seu modelo de negócio do envio de DVDs para streaming de vídeo, havia muitas dúvidas. Não existiam ferramentas comerciais disponíveis para ajudar a massiva infraestrutura em rede da empresa a rodar de forma suave. Assim, a Netflix decidiu se voltar para ferramentas open source. A empresa tomou a iniciativa de alistar a ajuda voluntária de centenas de desenvolvedores, criando com essa ajuda o "Exército Sírio", ou uma suíte de ferramentas automatizadas que testava a infraestrutura disponível, permitindo que a empresa fosse pró-ativa ao identificar e resolver vulnerabilidades antes que os clientes fossem impactados.

Desde então a Netflix continua a investir em automação e open source, e hoje, com o avanço de sua cultura DevOps, seus desenvolvedores integram e implantam código milhares de vezes ao dia.

A Fidelity Investments possuía várias unidades de negócio desenvolvendo software, além de muitas aplicações e processos legados que demandavam muito das equipes. As implantações eram manuais em centenas de servidores, gerando erros frequentes. Quando foi necessário desenvolver uma aplicação crítica de suporte a negociações sob um cronograma fixo, a organização aproveitou para adotar o DevOps e implementar um framework automatizado de liberação de software.

Tal solução resultou em mais de U$2.3 milhões por ano em economia de custos somente para o aplicativo que precisava ser desenvolvido. Desde então, dezenas de outros aplicativos tiveram suas liberações automatizadas, reduzindo o tempo necessário de três dias para uma ou duas horas, além de reduzir o downtime de teste. O processo também ajuda com as questões regulatórias, além de dar maior visibilidade às partes interessadas sobre os cronogramas de liberação.

Assista o vídeo entenda o que deve ser feito para implementar o DevOps.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Vantagens e desvantagens do DevOps

A adoção do DevOps pode por si só ser intimidadora. Com tanta coisa envolvida, incluindo tecnologias e perfis profissionais diversos, decisões, itens a serem priorizados, apoio institucional, apenas para citar alguns fatores, é compreensível que muitas organizações ainda não tenham dado os passos necessários para que esta adoção se concretize.

Em alguns dos casos, o incentivo para tomada de decisão pode vir de uma análise cuidadosa das vantagens e desvantagens do DevOps para o contexto da organização.

A seguir, apresentamos uma lista não-exaustiva das vantagens e desvantagens conhecidas trazidas pelo DevOps.

Vantagens

- Redução no tempo necessário para o desenvolvimento: com a aproximação entre desenvolvimento e operações, podemos esperar menores tempos necessários para aceitação e início do uso efetivo do software, tornando todo o processo mais econômico em termos de custos de produção. O tempo menor, bem como os custos menores, dependendo da situação (se a criação do software se destina ao consumo externo à organização), pode acabar por levar a uma fatia maior de participação no mercado.

- Redução no número de profissionais e custos associados: algumas estatísticas demonstram que é possível atingir uma redução de até 35% no número de profissionais de TI necessários para suportar o desenvolvimento e operações em organizações que usam DevOps. Isto é possível pelo forte uso da automação, bem como pela integração maior entre as equipes, reduzindo ruídos de comunicação e tornando o ciclo menos sujeito a erros. A redução geral em custos fica em torno de 30%.

- Possível endereçamento de questões relacionadas à segurança: dado que considerações acerca de segurança sejam trazidas para dentro do ciclo de desenvolvimento do produto, por meio da inclusão da integração e automação de práticas de segurança e procedimentos relacionados, organizações mais maduras fazem uso do chamado “DevSecOps”, ou seja, há integração e diálogo constantes entre desenvolvimento, segurança e operações.

- Auto-responsabilização dos membros da equipe: já que estes membros são incentivados a “empoderarem-se”, tomando para si mais responsabilidade sobre o produto, aumenta a capacidade da equipe como um todo a abraçar novas tecnologias, pois, ao interagirem mais com outros profissionais de outras equipes, tendem a absorver novos conhecimentos e se sentirem propelidos a inovar.

Desvantagens

- Problemas relacionados à segurança: este risco advém principalmente das situações em que as equipes de segurança não estão integradas ao grupo de DevOps, carecendo portanto de visibilidade sobre as abordagens usadas para acelerar o processo de desenvolvimento.

- Mudança cultural: o DevOps, no momento em que passa a ser adotado, requer grandes mudanças e adaptações na mentalidade e cultura da organização e dos funcionários. São mudanças que mexem com o status quo, muitas vezes construído ao longo da existência da organização e/ou da carreira destes indivíduos. Logo, não são mudanças fáceis de implementar. Resistências são esperadas, é preciso haver todo um trabalho de convencimento, se possível, baseado em fatos e números, para que a mudança de paradigma seja facilitada.

- Necessidade em lidar com experiências individuais históricas: A ligação entre DevOps e frameworks ágeis é inegável, e ambas privilegiam o emprego de profissionais com perfil autônomo e que tomam a iniciativa. Infelizmente, muitos profissionais vêm de situações anteriores que não privilegiavam este comportamento, logo eles têm dificuldade em entender que não existe mais separação entre desenvolvimento e operações. Ainda tendem a ver certas atividades como sendo Dev e outras atividades como sendo Ops. Esta linha de raciocínio não funciona no DevOps pleno, pois, ao lidar com um problema, o profissional precisa fazer sua própria investigação, já que não existe uma equipe de suporte separada. No máximo, o ticket existirá para dar suporte ao DevOps ao indicar e descrever toda a investigação que foi feita pelo próprio profissional, devendo o problema ser escalado horizontalmente (ou até mesmo verticalmente) apenas quando o profissional em questão realmente não possuir condições técnicas de endereçá-lo.

Os papéis relacionados ao DevOps

É importante diferenciar funções de papéis. Enquanto funções normalmente são formais, sendo o que define a relação da organização com o funcionário, papéis podem ser formais ou informais, sendo que um profissional com uma função específica pode ser revestido de um dado número de papéis.

A seguir, apresentaremos papéis que, no geral, fazem sentido à organizações que abraçam DevOps.



Atenção

É possível, portanto, que um profissional vista o chapéu de dois ou mais destes papéis, a depender de vários fatores, tais como a forma como a organização está estruturada, ou mesmo o número de profissionais envolvidos. Estes papéis também não podem ser considerados “canônicos”, ou seja, universalmente aceitos.

Como o DevOps em si é um *framework*, cada organização irá adotar os papéis individuais, as combinações de papéis e a denominação de papéis que fizerem sentido.

Evangelista

A adoção do DevOps requer planejamento, supervisão e promoção. Mudanças de hábito são necessárias, o que não irá se realizar a não ser que haja visibilidade clara dos benefícios oriundos dessas mudanças.

É aqui que o evangelista entra. Ele pode trabalhar como agente dessas mudanças para obter apoio das equipes de desenvolvimento e operação.

A pessoa que ocupar este papel pode identificar novos papéis que irão suportar a entrega e assegurar que os membros do time tenham acesso aos conhecimentos e recursos que eles precisam para completar a transição para uma cultura DevOps bem sucedida.

Gerente de liberação

Trabalham para coordenar e gerenciar o produto à medida que ele se move de desenvolvimento até a produção. Preocupados com a criação do produto, estão ainda mais focados em sua movimentação por meio do desenvolvimento, teste e implantação, de maneira que possam suportar a entrega contínua.

Isto significa que, por vezes, irão trabalhar em detalhes técnicos com os quais gerentes de projeto não se envolvem.

Arquiteto de automação

Também chamados de especialistas em integração, focam principalmente no desenho e implementação dos sistemas automatizados que são centrais ao componente da entrega contínua do DevOps.

Estes arquitetos desenham e implementam os sistemas automatizados que todos os membros da equipe DevOps irão usar durante o desenvolvimento e a produção.

Desenvolvedor/testador

É um papel expandido, já que não só existe codificação envolvida; no DevOps devem participar de testes unitários, implantação e monitoração contínua do produto. Esta mudança de mentalidade é construída sobre automações que fazem boa parte do papel pesado, mantendo ainda a agilidade esperada.

Profissional de garantia da experiência

Uma evolução da função da Garantia da Qualidade. Os profissionais da garantia da experiência procuram focar nas características e funcionalidades de novos produtos sempre sob a perspectiva da experiência do usuário.

Ao invés de simplesmente garantirem que algo funciona, procuram encontrar maneiras de otimizar e melhorar a experiência do usuário.

Engenheiro de segurança

Ao invés de ser uma preocupação secundária, no DevOps procura-se tratá-la como preocupação primária, incorporando seus elementos desde a concepção do produto.

O DevSecOps, como alguns preferem chamar esta prática, assegura que engenheiros de segurança trabalhem em conjunto com todo o time DevOps, integrando melhor estes princípios.

Assista o vídeo e conheça mais sobre o DevOps.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Verificando o aprendizado

Questão 1

Qual a ligação entre DevOps e *frameworks* ágeis?

A Os *frameworks* ágeis se originaram no DevOps.

B Os *frameworks* ágeis são o DevOps.

C O DevOps se originou nos *frameworks* ágeis.

D O DevOps é um *framework* ágil.

E Não há relação.

A alternativa C está correta.

O DevOps se originou nos *frameworks* ágeis, embora não seja por si só um framework ágil.

Questão 2

Qual indústria se encontra mais avançada em termos de adoção do DevOps?

A Manufatura.

B E-commerce.

C Saúde.

D Tecnologia.

E Finanças.

A alternativa D está correta.

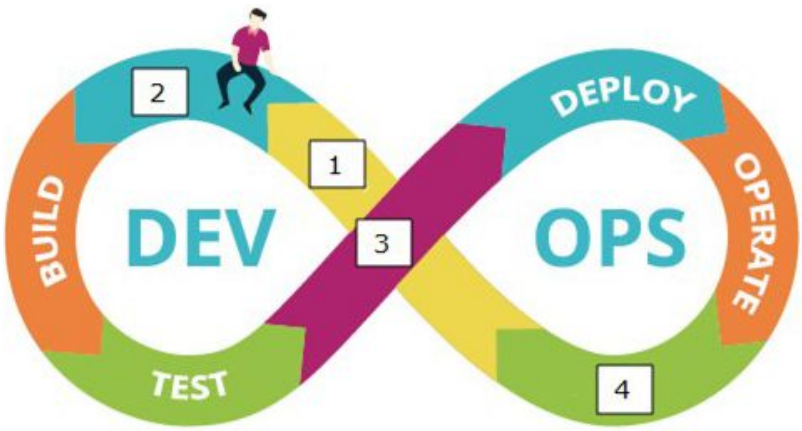
De acordo com Puppet, o segmento de tecnologia contava com cerca de 38% de adoção, bem à frente do segundo colocado, finanças, que detinha 12%.

Questão 3

Os itens que completam o ciclo de DevOps são, respectivamente:

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



A Prepare, Code, Quality Assurance, Monitor.

B Plan, Code, Release, Monitor.

C Plan, Develop, Release, Monitor.

D Proceed, Develop, Quality Assutance, Support.

E Proceed, Develop, Release, Support.

A alternativa B está correta.

Planejar - Codificar - Construir - Testar - Liberar - Implantar - Operar - Monitorar

Questão 4

Pela adoção do DevOps, em geral:

Objetivo: Identificar os papéis envolvidos e vantagens e desvantagens do DevOps

A Problemas de segurança, liberações, aumentados.

B Custos, problemas de segurança, reduzidos.

C Número de profissionais, mudanças, aumentados.

D Mudanças, liberações, reduzidos.

E Número de profissionais, custos, reduzidos.

A alternativa E está correta.

Algumas estatísticas demonstram que é possível atingir uma redução de até 35% no número de profissionais de TI e em torno de 30% nos custos gerais, dada a adoção do DevOps.

Questão 5

Qual dos papéis abaixo recebeu uma evolução direta com o DevOps?

A Auditor de configuração.

B Dono do processo de gerência de configuração.

C Gerente de projeto.

D Analista de qualidade.

E Analista de item de configuração.

A alternativa D está correta.

O profissional ou analista de experiência é uma evolução do analista de qualidade.

Considerações finais

Neste conteúdo, abordamos temas centrais da gerência de configuração moderna, com destaque para a integração contínua e o conceito de DevOps. Iniciamos pela explicação da integração contínua como prática essencial para automatizar e agilizar o desenvolvimento de software, promovendo a integração frequente de códigos por diferentes membros da equipe e reduzindo os riscos de conflitos e falhas. Essa abordagem permite identificar problemas ainda nas fases iniciais do processo, aumentando a confiabilidade das entregas e contribuindo para a melhoria contínua dos sistemas.

Apresentamos os principais desafios enfrentados na implementação da integração contínua e propusemos soluções práticas para superá-los, com foco especial na ferramenta Jenkins. Por sua ampla utilização no mercado, o Jenkins foi analisado como uma das soluções mais robustas para a automação de pipelines de integração e entrega contínuas, demonstrando sua aplicabilidade em diversos contextos de projeto.

Também exploramos o conceito de DevOps, destacando seu papel fundamental na transformação cultural e operacional das equipes de desenvolvimento e operações. A abordagem DevOps foi apresentada como um elo entre práticas técnicas e valores colaborativos, essenciais para acelerar entregas, aumentar a eficiência dos processos e garantir maior qualidade dos produtos finais.

Ao final deste estudo, reforçamos a importância da adoção dessas práticas para profissionais que atuam com desenvolvimento e manutenção de software, evidenciando como integração contínua, Jenkins e DevOps se inter-relacionam e contribuem significativamente para ambientes de trabalho mais produtivos, automatizados e resilientes.

Explore +

- Assista ao esclarecedor vídeo sobre integração contínua do canal Código Fonte TV.
- Assista o vídeo sobre DevOps do canal Dicionário do Programador.

Referências

- AIELLO, B. **Configuration Management Best Practices**. 1.ed. Pearson, 2013.
- BOURQUE, P.; FAIRLEY, R. **Software Engineering Body of Knowledge (SWEBOK)**. 3.ed. IEEE Computer Society, 2017.
- ERDER, M. **Continuous Architecture**. 1.ed. Morgan Kaufmann, 2015.
- HAAS, J. **Configuration Management Principles and Practice**. 1.ed. Addison Wesley, 2003.

- REED, M. **DevOps**: The Ultimate Beginners Guide to Learn DevOps Step-by Step. 1.ed., Produção Independente, 2019.

- SMART, J. **Jenkins The Definitive Guide Continuous Integration for the Masses**. 1.ed. O'Reilly, 2011.